

# Módulo 1 — Cómo Piensan los Modelos

---

## Fundamentos de IA Productiva

**Perfil del lector:** ingeniero informático con sólida experiencia técnica para quien el paradigma de inteligencia artificial es nuevo. Este documento usa analogías del día a día cuando son más claras que las técnicas, y recurre a conceptos de ingeniería solo cuando simplifican genuinamente.

---

## Tabla de Contenidos

1. ¿Qué es un LLM y cómo genera texto? - 1.1 La analogía del chef experimentado - 1.2 Tokens: la unidad mínima de trabajo - 1.3 El proceso de generación paso a paso - 1.4 ¿Qué hay dentro de la "caja negra"?
  2. El concepto de probabilidad y las alucinaciones - 2.1 Temperatura: el dial de la creatividad - 2.2 Por qué los modelos inventan cosas - 2.3 Tipos de alucinación y cómo reconocerlas
  3. Fortalezas reales y fallos sistemáticos - 3.1 Lo que los modelos hacen excepcionalmente bien - 3.2 Dónde fallan de forma predecible - 3.3 La regla práctica
  4. Qué diferencia a los modelos entre sí - 4.1 Tamaño: parámetros y lo que significan - 4.2 Entrenamiento base (pretraining) - 4.3 Fine-tuning: especialización post-fábrica - 4.4 RLHF: enseñar preferencias humanas - 4.5 Tabla comparativa de modelos actuales
  5. Cómo leer un benchmark - 5.1 Qué mide y qué no mide un benchmark - 5.2 Benchmarks comunes y su utilidad real - 5.3 La prueba definitiva: tu caso de uso
  6. Glosario del Módulo
  7. Preguntas de repaso
- 

## 1. ¿Qué es un LLM y cómo genera texto?

### 1.1 La analogía del chef experimentado

Un chef con 20 años de cocina no sigue un libro de recetas para improvisar un plato. Ha cocinado tanto, ha probado tantas combinaciones, que cuando le pides algo nuevo — "hazme algo con pollo, limón y jengibre" — lo crea sin haberlo visto antes. No inventa desde cero: combina patrones que absorbió durante años de práctica.

Un **LLM** (Large Language Model, modelo de lenguaje grande) funciona de forma análoga. En lugar de cocinar, leyó y procesó miles de millones de páginas de texto: libros, artículos, código, conversaciones. No memorizó cada página. Ajustó millones de valores internos llamados **pesos** para capturar los patrones del lenguaje. Cuando le escribes algo, usa esos patrones para predecir qué texto viene a continuación.

La diferencia clave con el autocompletado del celular: ese solo mira las últimas palabras que escribiste. Un LLM mantiene coherencia con todo el texto previo de la conversación y puede generar texto completamente nuevo que nunca existió antes.

**En resumen:** un LLM no busca respuestas en una base de datos ni sigue reglas escritas por humanos. Predice el texto más probable dada toda la conversación, usando patrones estadísticos aprendidos de una cantidad enorme de texto humano.

## 1.2 Tokens: la unidad mínima de trabajo

Un LLM no trabaja con letras individuales ni con palabras completas. Trabaja con **tokens** — fragmentos de texto que pueden ser una palabra completa, parte de una palabra, o un símbolo.

Piénsalo como las fichas de un juego de Scrabble: las palabras se arman combinando fichas, pero las fichas no siempre corresponden a sílabas naturales.

```
Texto: "La inteligencia artificial es fascinante"
Tokens: ["La", " intelig", "encia", " artific", "ial", " es", " fascinante"]
Cantidad: 7 tokens (aproximado, varía por modelo)
```

### ¿Por qué importa saber esto?

- Los modelos cobran por tokens (los que envías más los que generan).
- Tienen un límite máximo de tokens por conversación (la *ventana de contexto*, que veremos en el Módulo 2).
- Algunos errores curiosos — como que el modelo cuente mal las letras de una palabra — se explican por la tokenización: el modelo nunca ve "inteligencia" como una sola unidad, la ve como trozos.

## 1.3 El proceso de generación paso a paso

La generación de texto es un proceso donde el modelo produce un token a la vez, y cada nuevo token se añade al contexto para generar el siguiente.

Paso 1: Entrada → "El cielo es de color"

El modelo calcula probabilidades para todos los tokens posibles:

$P(\text{"azul"}) = 0.45$

$P(\text{"gris"}) = 0.18$

$P(\text{"rojo"}) = 0.12$

$P(\text{"verde"}) = 0.08$

... (miles de opciones más)

Paso 2: Se elige "azul" (el más probable, o uno de los probables con algo de azar)

Contexto nuevo → "El cielo es de color azul"

Paso 3: Se vuelven a calcular probabilidades para el siguiente token

$P(\text{"."}) = 0.35$

$P(\text{","}) = 0.15$

$P(\text{"y"}) = 0.20$

...

Paso 4: Se detecta el token de fin de secuencia → se detiene la generación

Lo que parece una respuesta fluida e instantánea es cientos o miles de estas predicciones encadenadas. El modelo no "piensa" la respuesta completa y luego la escribe — la construye token a token, sin poder revisar lo que ya generó.

---

## 1.4 ¿Qué hay dentro de la "caja negra"?

No necesitamos matemáticas para entender la arquitectura a nivel intuitivo. La arquitectura base se llama **Transformer** (2017). Sus dos mecanismos clave son:

### Mecanismo de atención (Attention)

Cuando estás en una conversación larga y alguien dice algo que solo cobra sentido si recuerdas un detalle mencionado hace 20 minutos, tu cerebro hace un esfuerzo para recuperar ese contexto relevante. El mecanismo de atención hace lo equivalente: al predecir el siguiente token, el modelo puede "mirar hacia atrás" en el texto y dar más peso a las partes que son relevantes para lo que está construyendo ahora.

### Capas de transformación

El modelo tiene muchas capas apiladas. Intuitivamente: las primeras capas detectan patrones simples (qué palabras tienden a aparecer juntas), y las más profundas detectan relaciones abstractas (quién hace qué a quién, cuál es el tono, cuál es la intención).

Lo importante para un usuario: el modelo no "entiende" el texto como lo hace un humano. Manipula representaciones matemáticas de las relaciones entre tokens. Que eso produzca resultados aparentemente comprensivos es sorprendente y real, pero el mecanismo subyacente es estadístico.

---

## 2. El concepto de probabilidad y las alucinaciones

### 2.1 Temperatura: el dial de la creatividad

Al elegir el siguiente token, hay un parámetro llamado **temperatura** que controla cuánta aleatoriedad se inyecta en esa elección.

Imagina el dial de temperatura de un horno, pero aplicado a la personalidad:

Distribución de probabilidades para el siguiente token:

|         |       |
|---------|-------|
| "azul"  | → 45% |
| "gris"  | → 18% |
| "rojo"  | → 12% |
| "verde" | → 8%  |
| "negro" | → 7%  |
| otros   | → 10% |

Temperatura = 0 (frío, determinista):

- Siempre elige "azul". Respuesta predecible y repetible.
- Como un funcionario que siempre sigue el reglamento al pie de la letra.

Temperatura = 0.7 (tibia, balanceada – el default habitual):

- Elige "azul" la mayoría de las veces, pero a veces "gris" o "rojo".
- Como un buen profesional: confiable pero con criterio propio.

Temperatura = 1.5 (alta, creativa):

- Las probabilidades se aplanan. Cualquier opción puede salir.
- Como un artista en modo improvisación: resultados sorprendentes, a veces brillantes, a veces incoherentes.

Para tareas que requieren precisión (código, datos, cálculos) se usa temperatura baja. Para tareas creativas (redacción libre, brainstorming) se usa alta. En interfaces como claude.ai el modelo elige el valor apropiado según el contexto.

---

## 2.2 Por qué los modelos inventan cosas

Las **alucinaciones** son respuestas que suenan plausibles y se expresan con confianza, pero son incorrectas, inventadas, o directamente falsas. Son el problema más importante que un usuario nuevo debe entender.

**Analogía:** imagina ese compañero de trabajo que nunca dice "no sé". Ante cualquier pregunta, responde con seguridad. A veces acierta porque realmente lo sabe. Pero cuando no sabe, en vez de admitirlo, improvisa una respuesta que suena razonable. El LLM tiene exactamente ese comportamiento por diseño.

### La causa técnica:

El modelo no accede a una base de datos de hechos verificados. Genera el token más probable dado el contexto. Si la pregunta es sobre un hecho poco representado en sus datos de entrenamiento, no tiene mecanismo para decir "no tengo esa información". Simplemente genera el texto que estadísticamente parece correcto en ese contexto.

```
Pregunta: "¿Cuál fue el resultado del partido entre Copiapó y O'Higgins  
el 14 de marzo de 2019?"
```

```
El modelo no tiene ese dato concreto.
```

```
Pero sabe que los resultados tienen la forma "X ganó Y-Z".
```

```
Genera: "Copiapó ganó 2-1 en el Estadio Municipal."
```

```
→ Completamente inventado. Presentado con total confianza.
```

---

## 2.3 Tipos de alucinación y cómo reconocerlas

| TIPO                 | DESCRIPCIÓN                                            | EJEMPLO                                                          |
|----------------------|--------------------------------------------------------|------------------------------------------------------------------|
| <b>Factual</b>       | Inventa datos concretos: fechas, cifras, nombres       | "La empresa fue fundada en 1987" (dato falso pero preciso)       |
| <b>Citación</b>      | Inventa referencias que no existen                     | Papers con título y autores plausibles pero inexistentes         |
| <b>Confabulación</b> | Mezcla hechos reales con inventados de forma coherente | Biografía de una persona real con eventos ficticios intercalados |
| <b>Razonamiento</b>  | Pasos intermedios incorrectos en una deducción         | Resultado correcto llegado por camino equivocado, o viceversa    |
| <b>Recencia</b>      | Datos desactualizados presentados como actuales        | "El CEO de la empresa es [nombre de hace 3 años]"                |

**Señales de alerta:** - El modelo da datos muy específicos (cifras exactas, fechas, nombres propios) sobre un tema oscuro o poco documentado. - Responde con la misma seguridad sobre algo que sabes con certeza y sobre algo que no puedes verificar. - Menciona fuentes sin que se las hayas pedido.

**Regla práctica:** cuanto más específico y verificable sea el dato, más vale confirmarlo por fuera. Para razonamiento general, síntesis y generación de texto, la tasa de error es baja. Para hechos puntuales, siempre verifica.

---

## 3. Fortalezas reales y fallos sistemáticos

### 3.1 Lo que los modelos hacen excepcionalmente bien

**Síntesis y comprensión de texto** Dado un documento largo, extraer los puntos clave, generar un resumen, responder preguntas sobre el contenido. Los modelos actuales son extraordinariamente buenos en esto, incluso con documentos muy técnicos.

**Transformación de formato** Convertir texto no estructurado a JSON, Markdown, XML, CSV. Reformatear una tabla. Extraer campos de un texto libre. Esto es muy fiable y ahorra tiempo enorme.

**Generación de código** Para la mayoría de lenguajes populares los modelos generan código funcional con alta frecuencia. El modelo ha "visto" millones de repositorios de código durante el entrenamiento.

**Reescritura y adaptación de tono** Tomar un texto técnico y hacerlo accesible para no técnicos, o viceversa. Adaptar un documento de estilo formal a uno conversacional. Los modelos hacen esto con alta calidad.

**Razonamiento sobre contexto dado** Si le das al modelo un documento con reglas, procedimientos o datos, y le pides que razone sobre ellos ("dado este contrato, ¿qué cláusula aplica a...?"), funciona muy bien. La clave es que la respuesta esté contenida en el contexto que le diste.

**Generación de variantes** Proponer 5 alternativas para un nombre de función. Reescribir un párrafo de 3 formas distintas. Generar casos de prueba para una función dada. Esta capacidad de generar variaciones a demanda es muy valiosa en el trabajo diario.

---

### 3.2 Dónde fallan de forma predecible

**Matemática y cálculos exactos** Los modelos son notoriamente malos con aritmética compleja, especialmente con números grandes o cálculos en múltiples pasos. No calculan: predicen el texto que parece el resultado de un cálculo. A veces coincide con la respuesta correcta; otras veces no.

Pregunta: ¿Cuánto es  $17 \times 83 + 44$ ?

→ Con números simples puede acertar, pero no porque calculó.

Lo predijo estadísticamente.

Pregunta: ¿Cuánto es  $7931 \times 6847$ ?

→ Aquí la probabilidad de error sube significativamente.

**Solución:** para cálculos, usa las herramientas integradas del modelo (ejecución de código) en lugar de confiar en la respuesta en texto.

**Conteo y operaciones sobre listas** "¿Cuántas veces aparece la letra 'a' en este párrafo?" es sorprendentemente difícil para un LLM. Recuerda: el modelo procesa tokens, no caracteres individuales.

**Conocimiento reciente o muy específico** Todo lo que ocurrió después de la fecha de corte del modelo es invisible para él. Y para eventos muy poco documentados en internet, la calidad decae — y aumenta el riesgo de alucinación.

**Instrucciones con muchas condiciones simultáneas** Cuantas más restricciones simultáneas impones en un solo prompt, más probable es que el modelo cumpla algunas y "olvide" otras. Un buen prompt es claro y no intenta hacer demasiado en una sola instrucción.

**Coherencia en documentos muy largos** Aunque las ventanas de contexto actuales son grandes, la atención a partes muy lejanas del contexto puede degradarse. Para documentos extensos, los detalles del inicio pueden "diluirse" hacia el final.

---

### 3.3 La regla práctica

Una heurística útil para decidir cuánto confiar en una respuesta:

| SI LA TAREA REQUIERE...                    | CONFIANZA RECOMENDADA                       |
|--------------------------------------------|---------------------------------------------|
| Síntesis de texto provisto                 | Alta                                        |
| Generación de código (lenguaje popular)    | Alta con revisión                           |
| Razonamiento sobre reglas explícitas dadas | Alta                                        |
| Hechos históricos bien documentados        | Media                                       |
| Hechos específicos recientes               | Baja — verificar                            |
| Cálculo numérico exacto                    | Baja — usar código                          |
| Conteo de caracteres o elementos           | Muy baja — verificar siempre                |
| Eventos oscuros o poco documentados        | Muy baja — alta probabilidad de alucinación |

**Principio clave:** el modelo es un colaborador muy capaz, no un oráculo. Trata sus respuestas como un borrador inteligente, no como una fuente de verdad. Tu rol es dar dirección y verificar lo que importa.

## 4. Qué diferencia a los modelos entre sí

### 4.1 Tamaño: parámetros y lo que significan

Los modelos se describen por su número de **parámetros** — los valores numéricos ajustados durante el entrenamiento. Se expresan en miles de millones: 7B, 13B, 70B, 405B.

**Analogía:** si el modelo es un cerebro, los parámetros son las conexiones entre neuronas. Más conexiones no garantiza más inteligencia, pero permite representar relaciones más complejas. Un modelo de 405B tiene físicamente más capacidad para capturar matices del lenguaje que uno de 7B.

**¿Más parámetros = mejor?** En general sí, pero con matices:

- Modelos más grandes requieren más hardware y son más lentos y costosos.
- Un modelo pequeño bien entrenado puede superar a uno grande entrenado descuidadamente en tareas específicas.



- Para la mayoría de usos cotidianos, un modelo mid-size es más que suficiente y notablemente más económico que el modelo más grande.

---

## 4.2 Entrenamiento base (pretraining)

El **pretraining** es la fase más costosa y lenta. Se expone al modelo a cantidades masivas de texto y se le entrena para predecir el siguiente token una y otra vez, miles de millones de veces, ajustando los pesos en cada iteración.

**Analogía:** imagina los primeros 22 años de vida de una persona: lee todo lo que cae en sus manos, escucha conversaciones, absorbe cultura. Al final sabe mucho sobre el mundo y el lenguaje. Pero todavía no sabe cómo ser útil en un trabajo específico — eso viene después.

Ejemplo simplificado de un paso de pretraining:

1. Texto del corpus: "La fotosíntesis convierte luz solar en [energía]"
2. El modelo predice el siguiente token → por ejemplo: "calor"
3. Se compara con el token real: "energía"
4. Se ajustan los pesos para reducir ese error
5. Se repite miles de millones de veces con textos distintos

Al final del pretraining el modelo sabe mucho, pero no es útil todavía: puede continuar cualquier texto, pero no sabe responder preguntas de forma estructurada ni seguir instrucciones.

---

## 4.3 Fine-tuning: especialización post-fábrica

El **fine-tuning** (ajuste fino) es un entrenamiento adicional, mucho más pequeño y económico, que toma el modelo base y lo especializa.

**Analogía:** es como la residencia médica. Un médico recién graduado (modelo base) tiene una formación amplia. Cuando entra a la residencia de cardiología (fine-tuning), no vuelve a empezar desde cero: profundiza en un área específica sobre la base que ya tiene. Al terminar, es el mismo médico pero mucho más experto en ese dominio.

### Tipos comunes:

**Instruction tuning:** se entrena con miles de ejemplos del formato pregunta/instrucción → respuesta. Esto es lo que transforma un "predicador de texto" en un "asistente que responde preguntas".

**Domain fine-tuning:** se entrena con textos del dominio específico (legal, médico, técnico). El modelo mejora en ese dominio concreto.

**Format fine-tuning:** se enseña al modelo a responder siempre en un formato específico (JSON, tablas, con etiquetas particulares).

El fine-tuning no crea conocimiento de la nada: si el modelo base no tiene conocimiento de un dominio, el fine-tuning no lo genera. Solo refuerza patrones que ya existen o ajusta el comportamiento y formato de las respuestas.

---

#### 4.4 RLHF: enseñar preferencias humanas

**RLHF** (Reinforcement Learning from Human Feedback) es la técnica que hace que los modelos sean útiles, seguros y alineados con lo que los humanos prefieren.

**Analogía:** imagina entrenar a un nuevo asistente dándole feedback constante. Le pides que redacte un correo, lo hace, y tú dices "este estuvo bien, directo y claro" o "este no, fue muy brusco y olvidó el contexto". Con suficientes iteraciones, el asistente aprende tu estilo sin que tengas que explicar cada regla. RLHF es exactamente ese proceso, pero a escala de millones de ejemplos.

El proceso en tres pasos:

**Paso 1 — Recopilar preferencias humanas** Se generan múltiples respuestas para la misma pregunta y evaluadores humanos dicen cuál prefieren y por qué. Se construye un dataset de comparaciones.

**Paso 2 — Entrenar un modelo de recompensa** Con esas preferencias, se entrena un modelo auxiliar (el *reward model*) que aprende a predecir qué respuestas gustarán más a los humanos. Actúa como un juez automático.

**Paso 3 — Ajustar el LLM con ese juez** El reward model puntúa las respuestas del LLM, y se ajustan los pesos del LLM para generar respuestas de mayor puntuación. Es un ciclo iterativo.

#### ¿Por qué importa para el usuario?

Es la razón por la que Claude o GPT responden de forma útil, declinan peticiones problemáticas, y tienen un "carácter" característico. También significa que diferentes modelos tienen personalidades distintas: cada empresa usa sus propios datos de preferencia.

---

4.5 Tabla comparativa de modelos actuales

| MODELO          | EMPRESA   | PUNTOS FUERTES                                   | CUÁNDO USARLO                                 |
|-----------------|-----------|--------------------------------------------------|-----------------------------------------------|
| Claude Opus 4   | Anthropic | Razonamiento profundo, análisis largo, escritura | Tareas complejas, flujos de trabajo autónomos |
| Claude Sonnet 4 | Anthropic | Equilibrio calidad/costo, velocidad              | Uso cotidiano, código, análisis               |
| Claude Haiku    | Anthropic | Velocidad, bajo costo                            | Tareas simples, alto volumen                  |
| GPT-4o          | OpenAI    | Multimodal sólido, ecosistema amplio             | Cuando se necesita visión integrada           |
| o3 / o4-mini    | OpenAI    | Razonamiento matemático y científico             | Problemas de lógica, matemática               |
| Gemini 2.5 Pro  | Google    | Contexto muy largo, multimodal                   | Análisis de documentos masivos                |
| Llama 3.x       | Meta      | Open source, desplegable localmente              | Privacidad, control total, sin costo API      |
| DeepSeek R1     | DeepSeek  | Razonamiento, open weights                       | Alternativa económica para razonamiento       |

**Nota:** el panorama cambia cada pocos meses. Un modelo que es el mejor en enero puede ser el tercero en abril. La comparación por benchmarks es útil como punto de partida, pero la prueba en tu caso de uso específico es siempre más informativa.

5. Cómo leer un benchmark

5.1 Qué mide y qué no mide un benchmark

Un **benchmark** es un conjunto de pruebas estandarizadas con respuestas conocidas, diseñado para medir el rendimiento de un modelo en condiciones controladas y comparables.

**Analogía:** es como las pruebas PISA o el examen de admisión a la universidad. Miden ciertas habilidades de forma estandarizada y permiten comparar. Pero un estudiante que saca 100 en el examen de admisión puede ser un profesional mediocre, y uno que sacó 80 puede ser brillante en su trabajo. El examen mide lo que el examen mide.

**Qué sí mide un benchmark bien diseñado:** - Rendimiento en ese tipo específico de preguntas, en ese momento - Comparación relativa entre modelos bajo condiciones iguales - Progreso de un modelo

entre versiones

**Qué no mide:** - Rendimiento en *tu* tarea específica - Calidad de la experiencia en conversación larga - Fiabilidad en dominios muy especializados o en español - Lo que importa para tu flujo de trabajo real

### 5.2 Benchmarks comunes y su utilidad real

| BENCHMARK             | QUÉ MIDE                                           | UTILIDAD PRÁCTICA                                               |
|-----------------------|----------------------------------------------------|-----------------------------------------------------------------|
| MMLU                  | Conocimiento general en 57 dominios                | Media — indica amplitud de conocimiento general                 |
| HumanEval / SWE-bench | Generación de código funcional, resolución de bugs | Alta para quienes programan                                     |
| MATH / AIME           | Problemas matemáticos de competencia               | Alta si necesitas razonamiento matemático                       |
| GPQA                  | Preguntas de nivel PhD en ciencias                 | Indica razonamiento profundo; poco relevante para uso cotidiano |
| MT-Bench              | Conversación multi-turno, instrucciones            | Media — mide bien la capacidad de asistente                     |
| Arena (LMSYS)         | Preferencias humanas en conversación libre         | Alta — refleja satisfacción de usuarios reales                  |

#### El problema del "benchmark hacking"

Existe evidencia de que los laboratorios optimizan sus modelos para los benchmarks públicos más conocidos, inflando artificialmente los resultados. Un modelo con 92% en MMLU no es necesariamente mejor para redactar propuestas comerciales que uno con 88%.

### 5.3 La prueba definitiva: tu caso de uso

La forma más fiable de elegir un modelo es construir un pequeño test personal: toma 5-10 tareas reales que necesitas hacer habitualmente, ejecuta las mismas tareas en los 2-3 modelos que consideras, y evalúa:

1. ¿Cuál resuelve mejor el problema sin instrucciones adicionales?
2. ¿Cuál produce el formato que necesitas?
3. ¿Cuál comete menos errores en tu dominio?
4. ¿Cuál es más conveniente en velocidad y costo para el volumen que usas?

Este proceso toma una hora y vale más que cualquier tabla de benchmarks.

## 6. Glosario del Módulo

| TÉRMINO                           | DEFINICIÓN BREVE                                                                                                       |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>LLM</b> (Large Language Model) | Modelo de lenguaje entrenado sobre grandes cantidades de texto para predecir el siguiente token                        |
| <b>Token</b>                      | Fragmento de texto que el modelo procesa como unidad mínima; puede ser una palabra, parte de una palabra, o un símbolo |
| <b>Parámetros</b>                 | Valores numéricos ajustables que definen el comportamiento del modelo; se miden en miles de millones (7B, 70B...)      |
| <b>Transformer</b>                | Arquitectura de red neuronal base de todos los LLMs modernos, introducida en 2017                                      |
| <b>Atención (Attention)</b>       | Mecanismo que permite al modelo relacionar partes lejanas del texto al predecir el siguiente token                     |
| <b>Temperatura</b>                | Parámetro que controla la aleatoriedad en la elección de tokens; 0 = determinista, >1 = muy creativo                   |
| <b>Alucinación</b>                | Respuesta incorrecta o inventada presentada con confianza por el modelo                                                |
| <b>Pretraining</b>                | Fase de entrenamiento masivo inicial sobre un corpus gigantesco de texto                                               |
| <b>Fine-tuning</b>                | Entrenamiento adicional sobre datos específicos para especializar el comportamiento del modelo                         |
| <b>RLHF</b>                       | Reinforcement Learning from Human Feedback; técnica para alinear el modelo con preferencias humanas                    |
| <b>Reward model</b>               | Modelo auxiliar que aprende a predecir qué respuestas prefieren los humanos; guía el RLHF                              |
| <b>Benchmark</b>                  | Suite de pruebas estandarizadas para medir y comparar el rendimiento de modelos                                        |
| <b>Autoregresivo</b>              | Proceso donde cada token generado depende de todos los tokens anteriores                                               |
| <b>Corpus</b>                     | El conjunto de textos usado para entrenar un modelo                                                                    |
| <b>Ventana de contexto</b>        | Cantidad máxima de tokens que un modelo puede procesar en una sola interacción (se detalla en Módulo 2)                |

---

## 7. Preguntas de repaso

Estas preguntas sirven para consolidar los conceptos antes de pasar al Módulo 2. No hay una respuesta "de examen": el objetivo es razonar en voz alta.

1. Si un modelo responde con una fecha exacta sobre un evento histórico menor, ¿qué deberías hacer antes de usar esa fecha en un documento importante?
2. Tienes que elegir entre usar un modelo de 7B que corre localmente en tu servidor y uno de 70B accesible por API. ¿Qué preguntas harías antes de decidir cuál usar para clasificar correos de clientes?
3. ¿En qué se parece el fine-tuning a una residencia médica? ¿Qué limitación comparten?
4. Un vendedor te muestra que su modelo tiene el puntaje más alto en MMLU. ¿Qué pregunta le harías para saber si eso es relevante para tu caso de uso?
5. Explícale a alguien sin contexto técnico por qué un LLM puede generar texto que nunca existió antes, si no tiene creatividad "real".

---

Siguiente: [Módulo 2 — Trabajar con el modelo](#)

---